



React Deep Dive — Complete Notes

Full 10-Day Curriculum — Every Concept, Every Example



Contents

1. Day 1 — useState Deep Dive
2. Day 2 — useEffect Deep Dive
3. Day 3 — useMemo, useCallback, memo
4. Day 4 — useReducer
5. Day 5 — Context Performance Splitting
6. Day 6 — Custom Hooks
7. Day 7 — Zustand
8. Day 8 — React Query
9. Day 9 — Concurrent React
10. Day 10 — Interview Q&A (Full 20)

DAY 1

useState — Stale Closures, Batching, Lazy Init

The most-asked hook; small mistakes here are the classic interview trap

1. WHAT IS A CLOSURE?

A closure is when a function "remembers" variables from where it was created, even after that outer scope has finished running.

```
function makeCounter() {  
  let count = 0  
  return function() {  
    console.log(count) // remembers count via closure  
  }  
}
```

Stale Closure — when the "memory" is outdated

A **stale** closure is when the remembered value is *old*, not the current one — the function captured a variable's value at creation time and never sees updates because the effect never re-runs.

```
// ❌ BUGGY – count is frozen at 0 forever  
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log(count) // always prints 0 – stale!  
    setCount(count + 1) // always sets to 0+1=1 – bug!  
  }, 1000)  
  return () => clearInterval(timer)  
}, [])
```

Mount → count=0 → effect runs → timer "photographs" count=0
1s later → timer fires → sees OLD photo (0) → sets count to 1

2s later → timer fires → sees SAME OLD photo (0) → sets count to 1 AGAIN
→ stuck at 1 forever

```
// ✅ FIXED – functional update asks React directly for latest value
useEffect(() => {
  const timer = setInterval(() => {
    setCount(c => c + 1) // c is always fresh from React
  }, 1000)
  return () => clearInterval(timer)
}, [])
```

Good closures vs bad (stale) closures

Situation	Closure type	Good or Bad
setCount(count+1) in a timer	Stale	Bad — need latest value
controller.abort() in cleanup	Intentional	Good — need old value to cancel right request
Button in a list closing over user.id	Intentional	Good — each item needs its own value
Debounce remembering typed text	Intentional	Good — remembers what triggered the delayed action

2. REACT 18 BATCHING

Multiple setState calls — even inside async functions — are merged into **ONE** re-render in React 18 (before React 18, async code did NOT batch).

```
const handleClick = () => {
  setCount(c => c + 1)
  setName("Suresh")
  // React 18 – only ONE render despite two updates
}

const handleAsync = async () => {
  await new Promise(r => setTimeout(r, 100))
  setCount(c => c + 1)
  setName("Suresh Babu")
  // React 18 – STILL only one render, even after an await
}
```

3. LAZY INITIALISATION

```
const getInitialValue = () => {
  console.log("expensive calculation runs...")
  return 100
}

// ❌ Runs getInitialValue() on EVERY render – wasted work
const [value, setValue] = useState(getInitialValue())

// ✅ Pass a FUNCTION – React calls it ONLY ONCE, on mount
const [value, setValue] = useState(() => getInitialValue())
```

Analogy: without lazy init you "cook the food then hand it to React" (already computed every render). With lazy init you "hand React the recipe" — React cooks it once, when it actually needs to.

4. COMMON ONCLICK BUG

`onClick={fn()}` **calls the function immediately during render**, not on click — this causes "Too many re-renders" infinite loop errors.

```
// ❌ Calls immediately – infinite loop  
<button onClick={set_value(v => v+1)}>
```

```
// ✅ Passes a reference – only runs when clicked  
<button onClick={() => set_value(v => v+1)}>
```

INTERVIEW ANSWER

"A stale closure happens when a function inside `useEffect` captures a variable's value at creation time and never sees updates because the effect doesn't re-run. The fix is functional updates like `setCount(c => c + 1)`, so React hands back the latest value directly instead of reading from a frozen closure. React 18 also batches all state updates — even inside async functions — into a single re-render."

DAY 2

useEffect — Cleanup, Race Conditions, When NOT to Use It

The most misunderstood hook in React

1. THREE SITUATIONS WHERE YOU SHOULD NOT USE USEEFFECT

Situation 1 — Deriving state

```
// ❌ Wrong – unnecessary useEffect
const [fullName, setFullName] = useState("")
useEffect(() => {
  setFullName(firstName + " " + lastName)
}, [firstName, lastName])

// ✅ Correct – just calculate it directly, no effect needed
const fullName = firstName + " " + lastName
```

Situation 2 — Filtering/transforming data

```
// ✅ Filter directly during render, no useEffect
const activeUsers = users.filter(u => u.active === true)
```

Situation 3 — Responding to user events

```
// ✅ Handle directly in the event handler
const handleSubmit = () => {
  console.log("form submitted!") // do it here, not via an effect watching state
}
```

2. RACE CONDITIONS — ABORTCONTROLLER

Two fetches firing (e.g. rapid clicks) can resolve *out of order* — an older, slower request can overwrite a newer, faster result with stale data.

```
useEffect(() => {
  const controller = new AbortController()
  setLoading(true)

  fetch(`/api/users/${userId}`, { signal: controller.signal })
    .then(res => res.json())
    .then(data => { setUser(data); setLoading(false) })
    .catch(err => {
      if (err.name === "AbortError") return // cancelled – normal, ignore
      console.error(err)
    })

  return () => controller.abort() // cancel PREVIOUS fetch when userId changes
}, [userId])
```

Click User 1 → fetch 1 starts

Click User 2 (fast) → cleanup runs → fetch 1 CANCELLED

→ fetch 2 starts

fetch 2 finishes → shows User 2 ✅ always correct, no race condition

3. THE CLEANUP FUNCTION — PROVING IT WITH CLOSURES

The cleanup closure **intentionally** remembers the OLD dependency value (e.g. old `userId`) so it can cancel exactly the right request — this is a GOOD use of closures, unlike Day 1's stale-closure bug.

```
return () => {
  console.log("cleanup – cancelling fetch for userId:", userId) // OLD userId, on purpose
}
```

```
controller.abort()  
}
```

4. DEPENDENCY ARRAY RULES

Array	Runs when
No array	After EVERY render (rarely wanted)
[]	Once, on mount only
[value]	On mount + whenever value changes

Forgetting [] on a fetch effect causes an infinite loop: render → effect runs → fetch → setState → re-render → effect runs again → forever.

5. TIMER ID (SETINTERVAL RETURN VALUE)

setInterval() doesn't return the timer itself — it returns a numeric ID assigned by the browser, like a ticket number, used later to cancel it via clearInterval(id).

INTERVIEW ANSWER

"useEffect is only for synchronising with external systems — APIs, timers, subscriptions — never for deriving state, filtering data, or responding to events. For fetches based on a changing value, I always use

AbortController in the cleanup function to cancel the previous request, which prevents race conditions where an older response overwrites a newer one."

DAY 3

useMemo, useCallback, React.memo

Reference equality — the root cause of unnecessary re-renders

1. REFERENCE EQUALITY

```
// Primitives – compared by VALUE
1 === 1           // ✅ true
"a" === "a"       // ✅ true

// Objects/Arrays/Functions – compared by REFERENCE (memory address)
{} === {}         // ❌ false
[] === []         // ❌ false
(() => {}) === (() => {}) // ❌ false
```

A new object/function literal created inline in a component body is a **brand-new reference every render** — this breaks `memo()` on children even when the actual data hasn't changed.

2. USEMEMO — CACHE A VALUE

```
const Parent = () => {  
  const [count, setCount] = useState(0)  
  const [name, setName] = useState("Suresh")  
  
  // ✅ same object reference unless name changes  
  const user = useMemo(() => ({ name }), [name])  
  
  return <Child user={user} /> // child skips render when only count changes  
}
```

When useMemo helps

- Genuinely expensive calculations
- Stable object references passed to memoized children
- Filtered/sorted derived lists

When useMemo HURTS

useMemo itself has a cost — storing the cached value and comparing dependencies every render. For cheap calculations (count * 2), plain calculation is *faster* than memoizing it.

3. USECALLBACK — CACHE A FUNCTION

```
// ❌ New function reference every render – breaks child's memo()  
const handleDelete = (task) => { setTasks(prev => prev.filter(t => t !== task)) }  
  
// ✅ Stable reference across renders  
const handleDelete = useCallback((task) => {
```

```
setTasks(prev => prev.filter(t => t !== task))
}, []) // setTasks is always stable, so [] is safe
```

4. MEMO() + USECALLBACK — THE FULL PATTERN

```
const TaskItem = memo(({ task, onDelete }) => {
  console.log("TaskItem rendered:", task)
  return (
    <div>
      {task} <button onClick={() => onDelete(task)}>Delete</button>
    </div>
  )
})

const TaskList = () => {
  const [tasks, setTasks] = useState(["A", "B"])
  const [count, setCount] = useState(0)

  const handleDelete = useCallback((task) => {
    setTasks(prev => prev.filter(t => t !== task))
  }, [])

  return (
    <>
      <button onClick={() => setCount(c => c+1)}>Count: {count}</button>
      {tasks.map(t => <TaskItem key={t} task={t} onDelete={handleDelete} />)}
    </>
  )
}
```

Result: clicking Count does **NOT** re-render any TaskItem — handleDelete stays the same reference (useCallback), so memo() correctly skips re-rendering the children.

5. HOW DOM REMOVAL ACTUALLY WORKS (WITH KEYS)

When you delete a task: setTasks updates state → React builds a new virtual DOM → React's diffing algorithm compares old vs new using the key prop → any old key missing from the new tree gets **unmounted** from the real DOM. Using array *index* as key can cause React to match the **WRONG** component after a deletion — always use a stable unique id/value as key.

INTERVIEW ANSWER

"useMemo caches a computed value; useCallback caches a function reference. Both prevent components wrapped in memo from re-rendering due to receiving a 'new' object or function reference every render, even when the underlying data hasn't actually changed. Overusing either has its own performance cost, so I only apply them for genuinely expensive work or to stabilise props passed into memoized children."

DAY 4

useReducer

Complex state, action-based updates, never mutate

1. BASIC COUNTER REDUCER

```
const countReducer = (state, action) => {
  switch (action.type) {
    case "increment": return state + 1 // ✅ new value
    case "decrement": return state - 1
    case "reset": return 0
    case "incrementBy": return state + action.payload
    default: return state // always handle default!
  }
}

const [count, dispatch] = useReducer(countReducer, 0)
dispatch({ type: "incrementBy", payload: 5 })
```

2. THE MUTATION BUG — CRITICAL

```
// ❌ WRONG – mutating then returning (dangerous pattern)
case "inc": return state += 1

// ✅ CORRECT – return a new value, never mutate
case "inc": return state + 1
```

With objects, mutation is even more dangerous: `state.name = "X"`; `return state` keeps the **same reference** — React thinks nothing changed and skips the re-render entirely.

3. OBJECT STATE REDUCER

```
const userReducer = (state, action) => {
  switch (action.type) {
```

```
case "updateName":  
  return { ...state, name: action.payload } // spread + override  
case "incrementAge":  
  return { ...state, age: state.age + 1 }  
case "reset":  
  return initialState  
default: return state  
}  
}
```

4. USEREDUCER VS USESTATE

Use useState when	Use useReducer when
Simple, independent values	Multiple related sub-values
Toggle, counter, single string	Complex transitions between states
Few update patterns	Many distinct action types

INTERVIEW ANSWER

"React detects state changes using reference equality. If I mutate an object directly and return the same reference, React thinks nothing changed and skips the re-render — so reducers must always return a brand new object using spread syntax, never mutate the existing state directly."

DAY 5

Context Performance Splitting

Why one combined context causes unnecessary re-renders — and the fix

1. THE PROBLEM

When state and dispatch live in **ONE** context, every consumer re-renders whenever *state* changes — even a component that only reads *dispatch*, which React guarantees never changes.

```
// TaskInput only needs dispatch, but still re-renders on every task added!  
const TaskInput = () => {  
  const { dispatch } = useAppContext() // combined context – wasteful  
}
```

2. THE FIX — TWO SEPARATE CONTEXTS

```
const AppStateContext = createContext(null)  
const AppDispatchContext = createContext(null)  
  
export const AppProvider = ({ children }) => {  
  const [state, dispatch] = useReducer(appReducer, initialState)  
  return (  
    <AppStateContext.Provider value={state}>  
      <AppDispatchContext.Provider value={dispatch}>  
        {children}  
      </AppDispatchContext.Provider>  
    </AppStateContext.Provider>  
  )  
}
```

```
    </AppStateContext.Provider>
  )
}

export const useAppState = () => useContext(AppStateContext)
export const useAppDispatch = () => useContext(AppDispatchContext)
```

3. APPLYING IT — DISPATCH-ONLY COMPONENTS NEVER RE-RENDER

```
const TaskInput = memo(() => {
  const dispatch = useAppDispatch() // ONLY subscribes to dispatch context
  // dispatch NEVER changes → TaskInput NEVER re-renders when tasks change ✓
})

const TaskList = () => {
  const state = useAppState() // only re-renders when STATE changes
}
```

4. COMPARISON — OLD VS NEW

Scenario	One combined context	Split contexts
Add a task	TaskInput re-renders too (wasted)	TaskInput skips render
Toggle theme	Task list re-renders too (wasted)	Only theme consumers re-render

INTERVIEW ANSWER

"Splitting state and dispatch into separate contexts means dispatch-only consumers never re-render, because dispatch is guaranteed stable by React across renders. Combined with memo on the components, this gives fine-grained control over re-renders without needing an external state library."

DAY 6

Custom Hooks — Built From Scratch

useFetch, useDebounce, useLocalStorage

1. USEFETCH

```
const useFetch = <T,>(url: string) => {
  const [data, setData] = useState<T | null>(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState<string | null>(null)

  useEffect(() => {
    const controller = new AbortController()
    const fetchData = async () => {
      try {
        setLoading(true)
        const res = await fetch(url, { signal: controller.signal })
        if (!res.ok) throw new Error(`HTTP ${res.status}`)
        setData(await res.json())
      } catch (err) {
        if (err.name !== "AbortError") setError(err.message)
      } finally {
```

```
        setLoading(false)
      }
    }
    fetchData()
    return () => controller.abort()
  }, [url])

  return { data, loading, error }
}
```

2. USEDEBOUNCE

```
const useDebounce = <T,>(value: T, delay: number): T => {
  const [debouncedValue, setDebouncedValue] = useState(value)
  useEffect(() => {
    const timer = setTimeout(() => setDebouncedValue(value), delay)
    return () => clearTimeout(timer) // cancel if value changes before delay finishes
  }, [value, delay])
  return debouncedValue
}
```

Without debounce: typing "Suresh" fires 6 API calls (one per keystroke). With debounce(500ms): only 1 API call fires, 500ms after the user stops typing.

3. USELOCALSTORAGE

```
const useLocalStorage = <T,>(key: string, initialValue: T) => {
  const [storedValue, setStoredValue] = useState<T>(() => {
```

```
    try {  
      const item = window.localStorage.getItem(key)  
      return item ? JSON.parse(item) : initialValue  
    } catch { return initialValue }  
  })  
  
  const setValue = (value: T) => {  
    setStoredValue(value)  
    window.localStorage.setItem(key, JSON.stringify(value))  
  }  
  
  return [storedValue, setValue] as const  
}
```

Persists across page refreshes — behaves exactly like `useState`, but backed by `localStorage` instead of memory.

4. TYPESCRIPT GENERIC SYNTAX NOTE

In `.tsx` files, `<T>` looks like a JSX tag to the parser. Fix: add a trailing comma `→ <T, >`, or use `<T extends unknown>`, or use the function keyword instead of an arrow function (no ambiguity there).

INTERVIEW ANSWER

"Custom hooks are functions starting with 'use' that encapsulate reusable stateful logic using other hooks. I create one whenever the same combination of hooks — like fetch + loading + error state — repeats across multiple components. It makes the code DRY, testable, and easier to read."

DAY 7

Zustand

Tiny global state library — no Provider, automatic selective re-rendering

1. BASIC STORE

```
import { create } from "zustand"

const useCounterStore = create((set) => ({
  count: 0,
  increment: () => set(state => ({ count: state.count + 1 })),
  decrement: () => set(state => ({ count: state.count - 1 })),
  reset: () => set({ count: 0 }),
}))

// use anywhere – no Provider wrapping needed!
const count = useCounterStore(state => state.count)
const increment = useCounterStore(state => state.increment)
```

2. SELECTIVE SUBSCRIPTION — THE KEY BENEFIT

Each component only re-renders when the *specific slice* of state it subscribed to changes — completely automatic, no manual context splitting needed like Day 5.

```
// This component ONLY re-renders when 'theme' changes – nothing else
const theme = useAppStore(state => state.theme)

// This component ONLY re-renders when 'tasks' changes
const tasks = useAppStore(state => state.tasks)
```

3. SLICES PATTERN (LARGER STORES)

```
interface TaskSlice {
  tasks: Task[]
  addTask: (text: string) => void
}

interface ThemeSlice {
  theme: "light" | "dark"
  toggleTheme: () => void
}

type AppStore = TaskSlice & ThemeSlice // combine multiple slices into one store

const useAppStore = create<AppStore>((set) => ({
  tasks: [],
  addTask: (text) => set(state => ({
    tasks: [...state.tasks, { id: Date.now(), text, completed: false }]
  })),
  theme: "light",
  toggleTheme: () => set(state => ({
    theme: state.theme === "light" ? "dark" : "light"
  }))
}))
```

```
    })),  
  }))
```

4. ZUSTAND VS CONTEXT VS REDUX

Feature	Context	Zustand	Redux
Setup	Provider + reducer + hooks	Just create()	Actions + reducers + store
Boilerplate	High	Very low	High (less with Toolkit)
Performance	Manual splitting needed	Automatic	Good
Provider needed	Yes	No	Yes
Best for	Simple shared state (theme, auth)	Most apps' global state	Huge enterprise apps

INTERVIEW ANSWER

"Zustand needs no Provider and gives automatic selective re-rendering — a component only re-renders when the specific slice of state it subscribed to actually changes. Context requires manually splitting state and dispatch into separate contexts to get the same effect, and Redux requires significantly more boilerplate for the same result."

DAY 8

React Query (TanStack Query)

Server state — caching, deduplication, background refetch, all built in

1. CLIENT STATE VS SERVER STATE

Client State	Server State
Theme, sidebar open/closed, form inputs	Users list, tasks, products — lives on a server
Tool: useState, Zustand, Context	Tool: React Query

2. WHY REACT QUERY OVER PLAIN USEEFFECT FETCHING

- **useEffect: no caching, no dedup, no retry, no background refetch, manual loading/error states, race conditions**
- **React Query: all of the above solved automatically, by queryKey**

3. USEQUERY — FETCHING

```
const { data: users, isLoading, isError, error, isFetching, refetch } = useQuery({
  queryKey: ["users"],           // cache identifier
  queryFn: fetchUsers,          // async function that fetches
  staleTime: 1000 * 30,         // fresh for 30 seconds
})
```

queryKey works like a cache key — same key returns cached data instantly; different key (e.g. ["user", 1] vs ["user", 2]) triggers a new fetch, but revisiting ["user", 1] later returns the cache instantly.

4. DYNAMIC QUERYKEY + ENABLED FLAG

```
const { data: user } = useQuery({
  queryKey: ["user", userId], // key changes → new fetch per userId
  queryFn: () => fetchUser(userId),
  enabled: userId > 0, // only fetch when a valid id is selected
})
```

5. USEMUTATION — CREATING/UPDATING/DELETING

```
const mutation = useMutation({
  mutationFn: (title) => fetch("/api/tasks", {
    method: "POST",
    body: JSON.stringify({ title })
  }).then(r => r.json()),

  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["tasks"] }) // auto-refetch fresh list
  }
})

mutation.mutate("New task") // trigger it
mutation.isPending // loading state, built in
```

6. STALETIME VS GCTIME

Setting	Meaning
staleTime	How long data is considered FRESH — no refetch during this window

gcTime	How long unused data stays in cache before being garbage collected
--------	--

7. ISLOADING VS ISFETCHING

- `isLoading` → true only on the very FIRST fetch (no cached data yet)
- `isFetching` → true on EVERY fetch, including silent background refetches

INTERVIEW ANSWER

"React Query solves problems `useEffect` fetching doesn't handle — caching by `queryKey`, deduplicating identical requests, retrying on failure, and refetching stale data in the background while still showing cached data instantly. `useMutation` handles create/update/delete, and calling `invalidateQueries` in `onSuccess` tells React Query to automatically refetch and update the UI."

DAY 9

Concurrent React

`useTransition`, `useDeferredValue`, `Suspense` + `lazy()`

1. USETRANSITION

Marks a state update as **non-urgent** — React finishes urgent updates (like typing) first, then processes the transition when it has spare time.

```
const [isPending, startTransition] = useTransition()

const handleChange = (e) => {
  setInput(e.target.value)      // urgent – updates instantly
  startTransition(() => {
    setSearch(e.target.value)    // non-urgent – React can delay this
  })
}

// isPending === true while the heavy update is still catching up
```

2. USEDEFERREDVALUE

```
const deferredSearch = useDeferredValue(search)
const isStale = search !== deferredSearch // true while list hasn't caught up yet
// input stays instantly responsive; heavy list re-renders when React has spare time
```

useTransition	useDeferredValue
You control WHEN a setState is marked non-urgent	You defer a VALUE you already have
Good when you own the state update	Good when value comes as a prop, no access to setState

3. SUSPENSE + LAZY() — CODE SPLITTING

```
// Without Suspense – everything loads upfront (slow initial load)
import HeavyComponent from './HeavyComponent'

// With Suspense – Loads ONLY when actually needed
```

```
const HeavyComponent = lazy(() => import("./HeavyComponent"))

const App = () => (
  <Suspense fallback={<p>Loading...</p>>}>
    <HeavyComponent />
  </Suspense>
)
```

Reduces initial bundle size — the component's code downloads only when it's actually rendered, not on first page load.

INTERVIEW ANSWER

"useTransition marks a state update as non-urgent so React prioritises things like input responsiveness first, then processes the heavy update when it has time — this keeps the UI from freezing. useDeferredValue achieves a similar effect but for a value directly, useful when you can't wrap the setState call yourself. Suspense with lazy() lets me code-split — components load on demand instead of bloating the initial bundle."

DAY 10

Top 20 Interview Questions — Full Answers

Rehearsed out loud, not just read

Q1 — What is the virtual DOM?

A lightweight JS copy of the real DOM. React builds a new virtual tree on state change, diffs it against the previous one, and updates only the changed parts in the real DOM — avoiding expensive full re-renders.

Q2 — What is a stale closure, and how do you fix it?

A function capturing a variable's value at creation time, never seeing updates because the effect doesn't re-run. Fix: functional updates like `setCount(c => c + 1)`.

Q3 — When should you NOT use `useEffect`?

Deriving state from props/state, filtering/transforming data, responding to user events — calculate directly or handle in the event handler instead.

Q4 — `useMemo` vs `useCallback`?

`useMemo` caches a computed VALUE; `useCallback` caches a FUNCTION reference — used with memoized children so they don't re-render on every parent render.

Q5 — Can `useMemo` make your app slower?

Yes — it has its own overhead (storing value, comparing deps). Only worth it for genuinely expensive calculations or stabilising references for memoized children.

Q6 — What is reference equality and why does it matter?

Primitives compare by value; objects/arrays/functions compare by memory reference. New objects/functions every render break `memo()` on children even when data is unchanged.

Q7 — `useReducer` vs `useState`?

`useState` for simple independent values; `useReducer` for complex related state, many action types, and testable state logic — pairs well with Context.

Q8 — Why never mutate state directly?

React detects changes via reference equality. Mutating keeps the same reference, so React thinks nothing changed and skips the re-render. Always return a new object with spread.

Q9 — Context performance problem and fix?

Combined state+dispatch context re-renders every consumer on any change. Fix: split into StateContext and DispatchContext — dispatch is stable, so dispatch-only consumers never re-render.

Q10 — Context vs Zustand?

Context needs a Provider and manual splitting for performance. Zustand needs no Provider and gives automatic selective re-rendering. Use Context for simple values, Zustand for complex global state.

Q11 — What is React Query and why use it?

A server state library solving caching, deduplication, retry, background refetch, and loading/error states automatically — all problems manual useEffect fetching has.

Q12 — What is staleTime?

How long fetched data is considered fresh — during this window, cached data is served with no network request; after it passes, React Query refetches in the background.

Q13 — What is useTransition for?

Marks state updates as non-urgent so React prioritises input responsiveness, processing the heavy update when it has spare time — keeps UI from freezing.

Q14 — useTransition vs useDeferredValue?

useTransition wraps the setState call itself; useDeferredValue wraps a value you already have — useful when you can't access the setState directly (e.g. value from a prop).

Q15 — What are custom hooks and when do you create one?

Functions starting with "use" encapsulating reusable stateful logic. Create one when the same hook combination repeats across components — DRY, testable, readable.

Q16 — What is the cleanup function in useEffect?

Runs before the next effect execution and on unmount — cancels timers, aborts fetches, removes listeners. Without it, timers/subscriptions keep running after the component is gone — memory leak.

Q17 — Why is the key prop important?

Helps React's reconciliation match old vs new virtual DOM nodes correctly during diffing. Without stable keys, React can re-render the wrong components or lose their state — never use array index for reorderable/filterable lists.

Q18 — What is Suspense?

Declaratively specifies a loading fallback for parts of the tree not yet ready — used with lazy() for code splitting, or with data fetching libraries to show a fallback while data loads.

Q19 — Client state vs server state?

Client state is app-owned data (theme, form inputs) — useState/Zustand. Server state lives on a server (users, tasks) with unique challenges (caching, staleness) — React Query is built specifically for this.

Q20 — How would you optimise a slow React app?

Profile with React DevTools → wrap expensive children in memo → wrap callback props in useCallback so memo works → wrap expensive calculations in useMemo → split Context into state/dispatch → use useTransition for

non-urgent heavy updates → use React Query to avoid redundant fetches → code-split with lazy + Suspense to shrink the initial bundle.

Built from your 10-day React deep dive · Full depth, not condensed · Keep alongside your EF Core / C# notes